

Python Programming

Unit 04 – Lecture 01 Notes

Tkinter Introduction and Widgets

Tofik Ali

February 20, 2026

Contents

1	Lecture Overview	1
2	Core Concepts	2
2.1	Basic Tkinter Program Structure	2
2.2	Widgets	2
2.3	Control Variables: <code>StringVar</code> , <code>IntVar</code>	2
2.4	The Event Loop	3
2.5	<code>ttk</code> Widgets	3
3	Demo Walkthrough	3
4	Interactive Checkpoints (with Solutions)	3
5	Practice Exercises (with Solutions)	3
6	Exit Question (with Solution)	4

1 Lecture Overview

Tkinter is Python’s built-in GUI toolkit. In this unit we learn to build desktop applications such as:

- calculators,
- forms (registration, login),
- and small utilities.

This lecture introduces the Tkinter window, the event loop, and commonly used widgets.

2 Core Concepts

2.1 Basic Tkinter Program Structure

Every Tkinter app has:

- a root window (`tk.Tk()`),
- widgets attached to the window,
- and an event loop (`mainloop()`).

```
import tkinter as tk

root = tk.Tk()
root.title("My App")
root.geometry("400x200")
root.mainloop()
```

2.2 Widgets

Widgets are UI components. Examples:

- Label: show text
- Button: clickable action
- Entry: single-line input
- Text: multi-line input/output
- Listbox: list selection
- Canvas: drawing
- Checkbutton, Radiobutton: choices

2.3 Control Variables: StringVar, IntVar

Tkinter provides variables that connect widget values to your code:

- StringVar for text
- IntVar for integers (often used for check/radio)

```
name_var = tk.StringVar()
entry = tk.Entry(root, textvariable=name_var)
```

Now you can read `name_var.get()` and set `name_var.set("...")`.

2.4 The Event Loop

`mainloop()` listens for events:

- button clicks,
- key presses,
- window close,
- etc.

Without `mainloop()`, your window appears briefly (or not at all) and the program ends.

2.5 ttk Widgets

`tkinter.ttk` provides themed widgets like `Combobox` (dropdown). They often look nicer and more modern.

3 Demo Walkthrough

File: `demo/widgets_showcase.py`

This demo shows multiple widgets in one window. Focus on:

- how widget values are read (`StringVar.get()`),
- how button callbacks work,
- and how selection widgets (check/radio/combobox) store values.

4 Interactive Checkpoints (with Solutions)

Checkpoint 1 Solution

Question: Why do we need `root.mainloop()`?

Answer: It starts the Tkinter event loop, keeps the window alive, and processes user events.

Checkpoint 2 Solution

Question: Benefit of using `StringVar` with `Entry`?

Answer: It provides an easy way to get/set widget values and keeps the UI and code connected.

5 Practice Exercises (with Solutions)

Exercise 1: Simple Greeting GUI

Task: Create an `Entry` for name and a `Button` that prints "Hello <name>".

Solution (idea):

```

import tkinter as tk

root = tk.Tk()
name = tk.StringVar()

tk.Label(root, text="Name:").pack()
tk.Entry(root, textvariable=name).pack()

def greet():
    print("Hello", name.get())

tk.Button(root, text="Greet", command=greet).pack()
root.mainloop()

```

Exercise 2: Checkbox + Radio

Task: Add one checkbox "Agree" and two radio options "Male/Female". Print selected values when a button is clicked.

Solution (outline):

```

agree = tk.IntVar()
gender = tk.StringVar(value="Male")

tk.Checkbutton(root, text="Agree", variable=agree).pack()
tk.Radiobutton(root, text="Male", variable=gender, value="Male").pack()
tk.Radiobutton(root, text="Female", variable=gender, value="Female").pack()

```

6 Exit Question (with Solution)

Question: 3 lines to create a window titled "Hello GUI" and keep it running.

Solution:

```

root = tk.Tk()
root.title("Hello GUI")
root.mainloop()

```

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Create a basic Tkinter window and run the GUI loop
- Understand widgets and how they are placed in a window
- Use common widgets: Label, Button, Entry, Text, Canvas, Listbox
- Use selection widgets: Checkbutton, Radiobutton, Combobox, Spinbox

What is Tkinter?

- Tkinter is Python's standard GUI library
- Allows building desktop apps (forms, tools, mini-projects)
- Works with widgets (buttons, text boxes, etc.)

Smallest Tkinter App

```
import tkinter as tk

root = tk.Tk()
root.title("My App")
root.geometry("400x200")
root.mainloop()
```

- Tk() creates the main window
- mainloop() keeps the window running

Widgets (GUI Building Blocks)

- Widgets are UI components: Label, Button, Entry, etc.
- Most widgets need:
 - a parent (usually root or a Frame)
 - configuration options (text, width, command, ...)

Label, Button, Entry (Example)

```
import tkinter as tk

root = tk.Tk()
name_var = tk.StringVar()

tk.Label(root, text="Name:").pack()
tk.Entry(root, textvariable=name_var).pack()

def greet():
    print("Hello,", name_var.get())

tk.Button(root, text="Greet", command=greet).pack()
root.mainloop()
```

More Widgets

- Text: multi-line text input/output
- Canvas: drawing shapes
- Scrollbar: scrolling support (Text/Listbox)
- Listbox: list selection

Selection Widgets

- Checkbutton: multiple independent selections
- Radiobutton: choose exactly one option
- ttk Combobox: dropdown list
- Spinbox: numeric selection

Demo: Widget Showcase

- File: demo/widgets_showcase.py
- Shows:
 - Label, Button, Entry
 - Checkbutton and Radiobutton
 - Listbox and Text
 - ttk Combobox

Checkpoint 1

Question: Why do we need `root.mainloop()` in Tkinter?

Checkpoint 2

Question: What is the benefit of using `StringVar` with an `Entry` widget?

Think-Pair-Share

Discuss:

- What makes a GUI *easy to use*?
- Give 3 design rules (labels, spacing, validation, feedback, etc.).

Key Takeaways

- Tkinter builds GUI apps using widgets
- `Tk()` creates a window; `mainloop()` runs the event loop
- Widgets like `Label`/`Button`/`Entry` handle input and actions
- Selection widgets (`check`/`radio`/`combobox`) capture choices

Exit Question

Write the 3 key lines needed to create a Tkinter window titled "Hello GUI" and keep it running.